Basics Types and Variables

L07 - Types, Variables and Input

Department of Computer Science
University of Pretoria

05 March 2024

Overview

A type provides the blue-print to which our variables and constants that we create within our program need to adhere. Types in C++ are categorised as Primitive (or built-in), *Derived and Abstract or (user-defined).*

Primitive Types

Primitive types fall into two main categories. Types where the **previous** *and* **next values** of the type are **predefined** and types where the previous and next values could respectively be any value **smaller or greater than the current value.**

Values that are **predefined** and therefore **predictable** are said to be **ordinal**, while values where we **cannot determine** the next value in the **sequence are values of variable-length types**.

1. Ordinal

Ordinal types are Integer, Character and Boolean. The keywords used to represent these types are int, char and bool respectively.

2. Variable-length

Variable-length types include floating point, of which there are two types: float and double. The difference lies in precision.

Floating point data type stores single-precision floating-point values, with float typically requiring 4 bytes of memory space.
Double floating point, on the other hand, stores double-precision floating-point values and typically requires 8 bytes of memory space.

3. Type modifiers

In C++ type modifiers are used to change the properties of basic data types, so that programmers can define variables more suited to the specific application. These modifiers include: • signed (for int and char) • unsigned (for int and char) • short (for int) • long (for int and double)

1. Signed: Can store positive and negative numbers. It does not change the storage size, but halves the positive range to accommodate negative values. The default for C++ integers.
2. Unsigned: Can only store positive numbers and zero, doubling the positive range by eliminating the need for bits to represent negative values.

3. Short: Reduces storage size and range of values. Less memory usage, but also less range.
4. Long: Increases storage size and range of values. More memory usage, but also a greater range.

Type	C++ type name	Byte size	Range
Ordinal	`int`	4	-2147483648 → 2147483647
	`char`	1	-128 → 127
	`bool`	1	`true` or `false`
Variable-length	`float`	4	$-3.4 \times 10^{38} \rightarrow 3.4 \times 10^{38}$
	`double`	8	$-1.7 \times 10^{308} \rightarrow 1.7 \times 10^{308}$
Variable-length (Library-based)	`std::string`	24	
Modifiers	`short int`	2	-32768 → 32767
	`unsigned short int`	2	0 → 65535
	`unsigned long int`	4	0 → 4294967295
	`long long int`	8	$-2^{63} \rightarrow 2^{63}-1$
	`long double`	12	$-1.1 \times 10^{4932} \rightarrow 1.1 \times 10^{4932}$

Table 3.1: Basic and Modified Types

1. Variables

A variable is a letter or word that can stand in for any number, character, string, list,etc..
Think of a variable as a place-holder for a value.
Variables in C++ are defned by providing a type and name for the variable in the format: (typename)(variablename)

(typename)(variablename)
where **typename** is one of the types given in Table 3.1 and **variablename** the name the variable will be referred to as in the progoram.

Once a variable has been defined, a value can be assigned to the variable.

An example of a definition of a variable is given by:
int radius ;
(a variable with the name radius is defined of type int).
This means radius may take on values that are integers.

Rules for declaring variables:

1. A variable name can consist of Capital letters A–Z, lowercase letters a–z digits 0–9, and the underscore character.
2. The first character must be a letter or underscore.
3. Blank spaces cannot be used in variable names.
4. Special characters like # and $ are not allowed.
5. C++ keywords cannot be used as variable names.
6. Variable names are case–sensitive.
7. Variable type can be bool, char, int, float, double, void

2. Constant

Constants are used as a naming mechanism for values that may not change during the life of a program.

Typical values that can be defined as a constant are PI, MAX STUDENTS,etc

These values are defined once and can be used a number of times within the program.

There are two ways that a constant:

1. Preprocessor method

A constant defined using the preprocessor method is defined by using the keyword define, providing a name for the constant and a corresponding value.

As the constant is defined outside the main function (and later you will see, outside any other function),available to all parts.

The syntax for defining a constant using the preprocessor method is given by:

```
#define <CONSTANTNAME> <constant value>
```

Below is an example to define PI.

```
#define PI 3.1415
```

Program illustrates where and how the constant is defined and used

```
#include <iostream>
#define PI 3.1415
using namespace std;

int main ( )
{
cout << "The area of a circle with a r adius of 10cm is : "
<< PI * 10 * 10 ;
return 0 ;
}
```

2. const keyword method

Defining a constant using the keyword method is similar to defining a variable in C++.

To define a constant the keyword const must be specified followed by the definition of constant. A constant is identified by a name and a type.

The syntax to define the constant is given by:
const <typename> <CONSTANT_NAME> = <constant_value >;

For example, the constant value 10 is named RADIUS and is of type int in the example below. Rather than using 10 in the code when referring to the radius. const int RADIUS = 10 ;

Depending on where a constant in defined in a C++ program, will determine the scope of the constant.

```
#include <iostream>
const float PI = 3.1415;
using namespace std;

int main ( )
{
const int RADIUS = 10 ;
cout<< "The area of a circle with a radius of 10cm is: "
    << PI * RADIUS * RADIUS;
return 0 ;
}
```

3. Scope

Defining the constant outside the main function enables the constant to be used globally.

If the constant is defined within the main function, only the main function knows about the constant and can use the constant.

4. Naming conventions

It is important that when you choose a name for a variable or constant, it is descriptive.
This will make it easier to read the code.
Code that is easier to follow is easier to maintain.
The variable name must begin with a letter and may have no spaces.

You will see that throughout this text, variables start with small letters and when a variable consists of more than one word, the other words start with capital letters.

This is merely a convention we follow for the sake of readability e.g. drawDoubleStoryHouse is more readable than drawdoublestoryhouse.

Constant names are typically written in all capital letters to easily distinguish them from variables.

The variable type and the keyword constant type is any valid type defined by the C++ language or the programmer.

Refer to Table 3.1. The preprocessor constant type is determined by the compiler based on the value given to the constant.

Variables can be initialized when declared:
int first=13, second=10;
char ch= 'R';
string color = "blue " double x=12.6; All variables must be initialized before they are used But not necessarily during declaration

Int data type

Examples:

-6728

0

78

+763 Positive integers do not need a + sign,

No commas are used within an integer

Commas are used for separating items in a list

Floating-Point Data Types

C++ uses scientific notation to represent real numbers (floating-point notation)

TABLE 2-3 Examples of Real Numbers Printed in C++ Floating-Point Notation

Real Number	C++ Floating-Point Notation
75.924	7.592400E1
0.18	1.800000E-1
0.0000453	4.530000E-5
-1.482	-1.482000E0
7800.0	7.800000E3

1. **float:** represents any real number

Range: -3.4E+38 to 3.4E+38 (four bytes)

2. **double:** represents any real number

Range: -1.7E+308 to 1.7E+308 (eight bytes)

On most newer compilers, data types double and long double are same

Char data type:

The smallest integral data type

Used for characters: letters, digits, and special symbols

Each character is enclosed in single quotes

'A', 'a', '0', '*', '+', '$', '&'

A blank space is a character and is written ' ',

with a space left between the single quotes

string Type

Sequence of zero or more characters

Enclosed in double quotation marks.

Example: string color = "red"

Null: a string with no characters. Example: string color = "" ""

Each character has relative position in string

-Position of first character is 0

bool Data Type

Two values: true and false

Manipulate logical (Boolean) expressions

true and false are called logical values

bool, true, and false are reserved words

In C++, type conversions, also known as typecasting or coercion, refer to the process of converting one data type into another.

Type conversions are essential for ensuring compatibility between different data types in expressions, assignments, and function calls. C++ supports several mechanisms for type conversions, namely:

1. Implicit Conversions

Implicit conversions are performed automatically by the compiler when it encounters expressions involving different data types.

2. Explicit Conversions (Typecasting)

Also known as typecasts, are performed by the programmer to explicitly convert a value from one data type to another.

EXAMPLE 2-9

Expression	Evaluates to
`static_cast<int>(7.9)`	7
`static_cast<int>(3.3)`	3
`static_cast<double>(25)`	25.0
`static_cast<double>(5+3)`	= `static_cast<double>(8)` = 8.0
`static_cast<double>(15) / 2`	= 15.0 / 2 (because `static_cast<double>(15)` = 15.0) = 15.0 / 2.0 = 7.5
`static_cast<double>(15 / 2)`	= `static_cast<double>(7)` (because 15 / 2 = 7) = 7.0
`static_cast<int>(7.8 +` `static_cast<double>(15) / 2)`	= `static_cast<int>(7.8+7.5)` = `static_cast<int>(15.3)` = 15
`static_cast<int>(7.8 +` `static_cast<double>(15 / 2))`	= `static_cast<int>(7.8 + 7.0)` = `static_cast<int>(14.8)` = 14

```
#include <iostream>

int main() {
    // Explicit type casting
    double x = 3.14;
    int y = static_cast<int>(x);

    std::cout << "Double value: " << x << std::endl;
    std::cout << "Int value after casting: " << y << std::endl;

    // Implicit type casting
    int a = 10;
    double b = a; // Implicit conversion from int to double

    std::cout << "Int value: " << a << std::endl;
    std::cout << "Double value after implicit casting: " << b << s

    return 0;
}
```

cin is used with >> to gather input
cin>>variable>>variable

The stream extraction operator is >>
For example, if miles is a double variable
cin >> miles;
Causes computer to get a value of type double
Places it in the variable miles

EXAMPLE 2-17

```
#include <iostream>

using namespace std;

int main()
{
     int feet;
     int inches;

     cout << "Enter two integers separated by spaces: ";
     cin >> feet >> inches;
     cout << endl;

     cout << "Feet = " << feet << endl;
     cout << "Inches = " << inches << endl;

     return 0;
}
```

Sample Run: (In this sample run, the user input is shaded.)

```
Enter two integers separated by spaces: 23 7

Feet = 23
Inches = 7
```